

Cloud Computing

UNIT-5

Python Web Application Framework

Django

- It is a open source framework for developing web applications in python.
- Django is based on the **Model-Template-View** architecture and provides a separation of the data model from the business model and the user interface.
- Django can work with different databases with out changing any code.

Django Architecture

Model

- The model acts as a definition of **some stored data** and handles the interactions with the database.
- In a web application data can be stored in a relational or non-relational databases or an XML files.
- Model is a python class that outlines the variable and methods for a particular type of data.

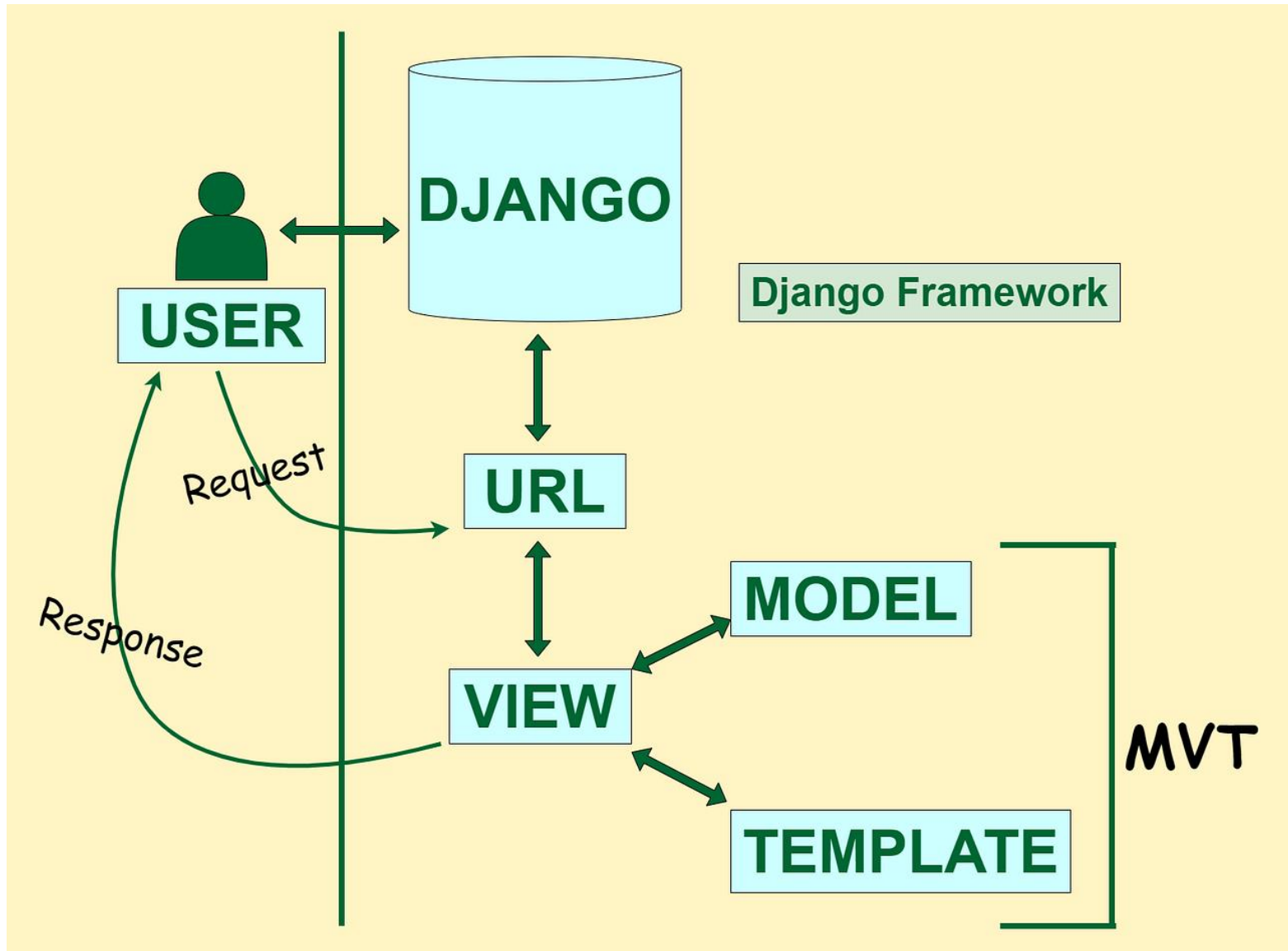
Python Web Application Framework

Template

- In a typical Django web application the template is simply a HTML page with a few extra place holders.
- Django template language can be used to create various forms of text files(XML,email,CSS,JavaScript,CSV etc).

View

- The view ties the model to the template.
- The view is where you write the code that actually generate the web pages.
- View determines what data is to be displayed, retrieves the data from the database and pass the data to the template.



Design Approaches

Design methodology for IaaS service model

- Traditional application design approaches like service-oriented architectures(SOA) use component based design.
- **Cloud Control Matrix (CCM)** approach for designing cloud applications, which is more recent approach that classifies the components based on the types of functions performed and types of cloud resources used.
- The CCM design approach is suited for applications that use the IaaS cloud service model.
- With the IaaS model the developers get the flexibility to map the CCM architectures to cloud deployments.
- Virtual machines for various tiers can be provisioned and auto scaling options for each tier can be defined.

a. Component Design

- CCM is created for the application based on **comprehensive analysis** of the applications functions and building blocks.
- CCM allows identifying the building blocks of a cloud application which are classified based on the functions performed and type of cloud resources required.
- Each building block performs a set of actions to produce the desired outputs for other components.
- **Each component takes a specific inputs**, performs a pre-defined set of actions and produces the desired outputs.
- Components offer their functions as service through a functional interface which can be used by other components.
- Components report their performance to a performance database through a performance interface.
- Components have number of resources like web pages, images, documents, database tables etc.

b. Architecture Design

- In the Architecture design interactions between the application components are defined.

Characteristics of CCM components:

1. Loose Coupling :

- Components in the CCM are loosely coupled.
- Instead of hard-writing the links, the components interface through clearly defined functional and service boundaries.
- Loose coupling of components relies on the use of REST communication protocol that allows components developed in different programming languages to communicate with each other.

2. Asynchronous communication:

- Loosely coupled components communicate asynchronously through message based communication.
- It isolates all components so all components interact asynchronously by treating other components as black box.

- By allowing asynchronous communication between components, it is possible to add capacity by adding additional servers when the application load increases.
- Asynchronous communication is made possible by using messaging queues.
- The benefit of messaging queues is that the overall application can continue to perform even through individual components may go offline temporarily.
- If a component become temporarily un available, the messages are buffered and processed when the components become available.

3.Stateles Design:

- CCM components are stateless.
- By storing session state outside of the component, stateless component design enables distribution and horizontal scaling.
- In distributed computing successive requests to a component may be serviced by different servers.

c. Deployment Design

- In this application components are mapped to specific cloud resources such as web servers, database servers.
- All application components are designed loosely coupled and asynchronous communication, components are deployed independently of each other.
- Multiple clouds can be used for application deployment.
- This approach makes it easy to migrate application components from one cloud to another.
- The application developer can ensure that the applications meet the performance and cost requirements with the changing context.

Benefits of CCM Approach :

Improved Application Performance

- CCM uses loosely coupled components that communicate asynchronously. So that an application can scale up and scale down independently.

Savings in Design, Testing and Maintenance Time

- It uses standard components and it uses loosely coupled components which can be tested independently.

Reduced Application Cost

- CCM design approach allows the components are loosely coupled, so that the both scaling allows to scale up the needed resources.

Reduced Complexity

- CCM provides both types of scaling options based on the requirement of the application.

Design Methodology for PaaS service model

- For **Applications that use the PaaS cloud service model**, the architecture and deployment design steps are not required because the platform takes care of the architecture and deployment.
- In the **component design step**, the developers have to take into **consideration the platform specific features**.
- Applications designed for **specific PaaS offerings run in sandbox environments** and are allowed to perform only those actions that do not interfere with the performance of other applications.
- The deployment and scaling is handled by the platform while the developers focus on the application development using the platform-specific SDKs.
- Portability is a major constraint for the PaaS based applications as it is difficult to move the application from one cloud to another cloud vendor.

Image Processing Application

- It provides online image filtering capability.
- Users can upload image files and choose the filters to apply.
- The selected filters are applied to the image and the processed image can be downloaded.
- In the component design step we identify the application components and group them based on the type of functions performed and type of resources required.
- The web tier for the image processing app has front ends for image submission and displaying processed images.
- The application tier has components for processing the image submission requests, processing the submitted image and processing requests for displaying the results.
- The storage tier comprises of the storage for processed images.

Image Processing Application

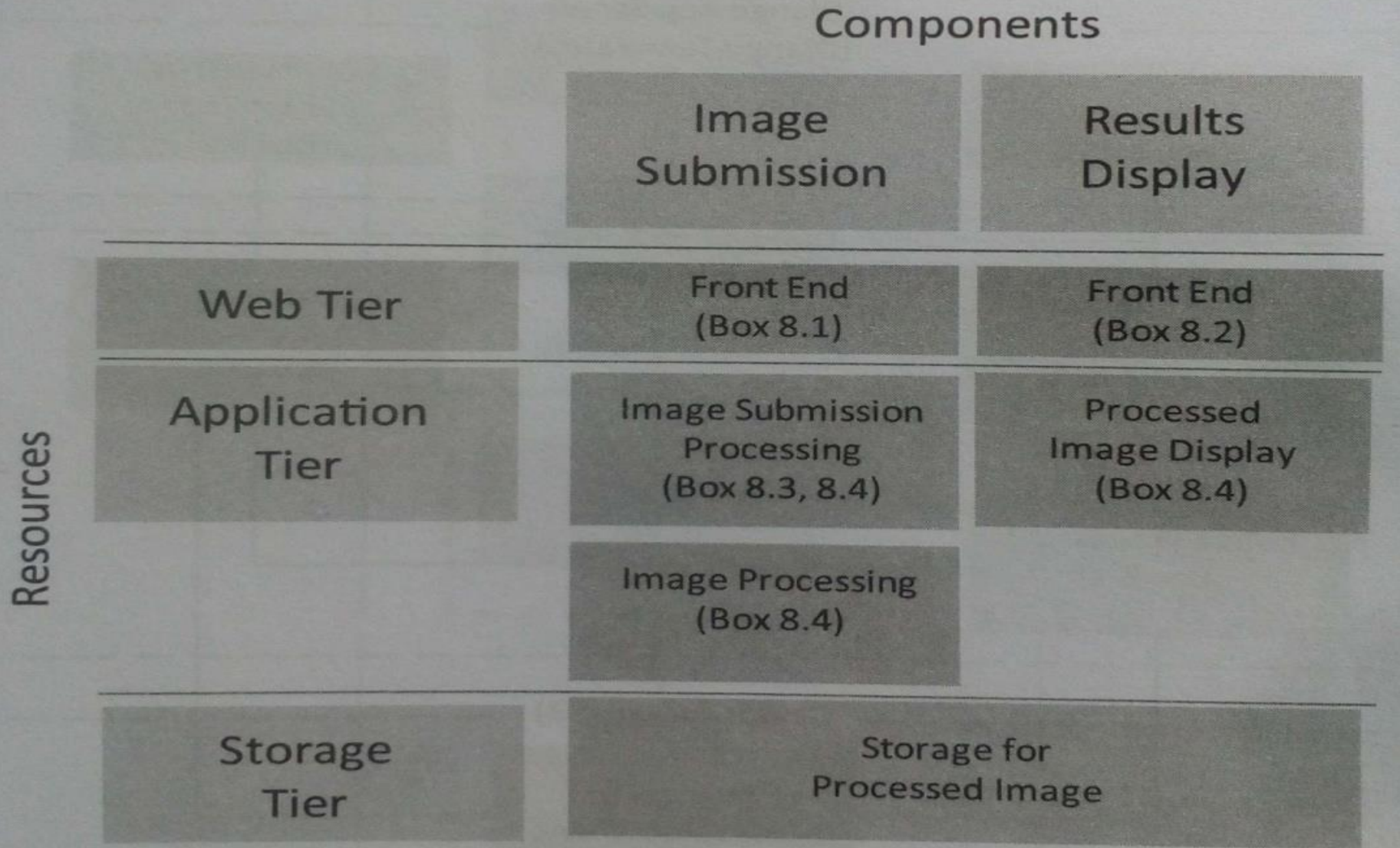


Fig : Component Design for Image Processing App

Image Processing Application

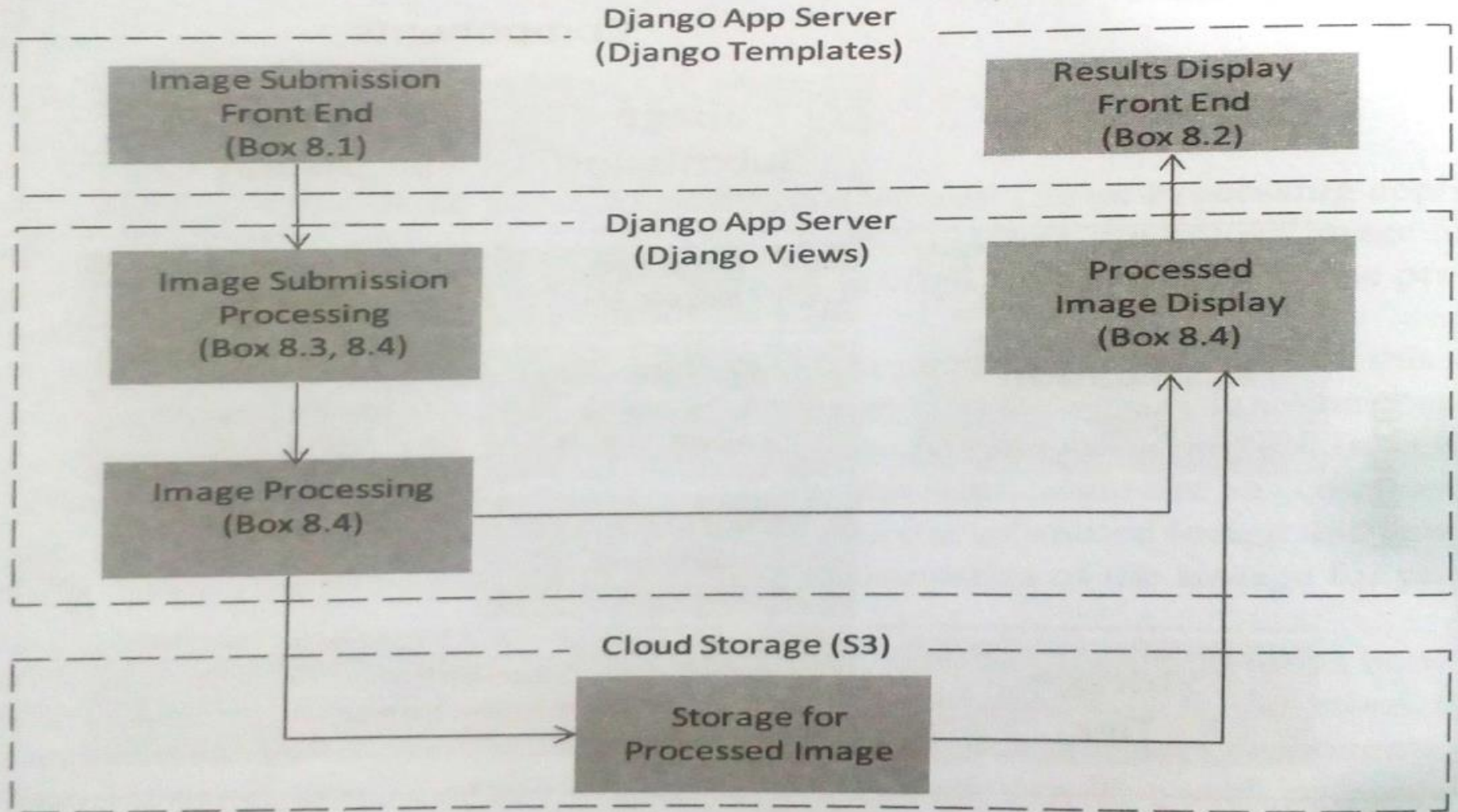


Fig : Architecture Design for IPA

Image Processing Application

- In the architectural design step defines the interactions between the application components.
- The web tier components map to the Django templates and the application tier components map to the Django views.
- A cloud storage is used for the storage tier.
- Deployment design is a multitier architecture comprising of load balancer, application server and a cloud storage for processed images.

Image Processing Application

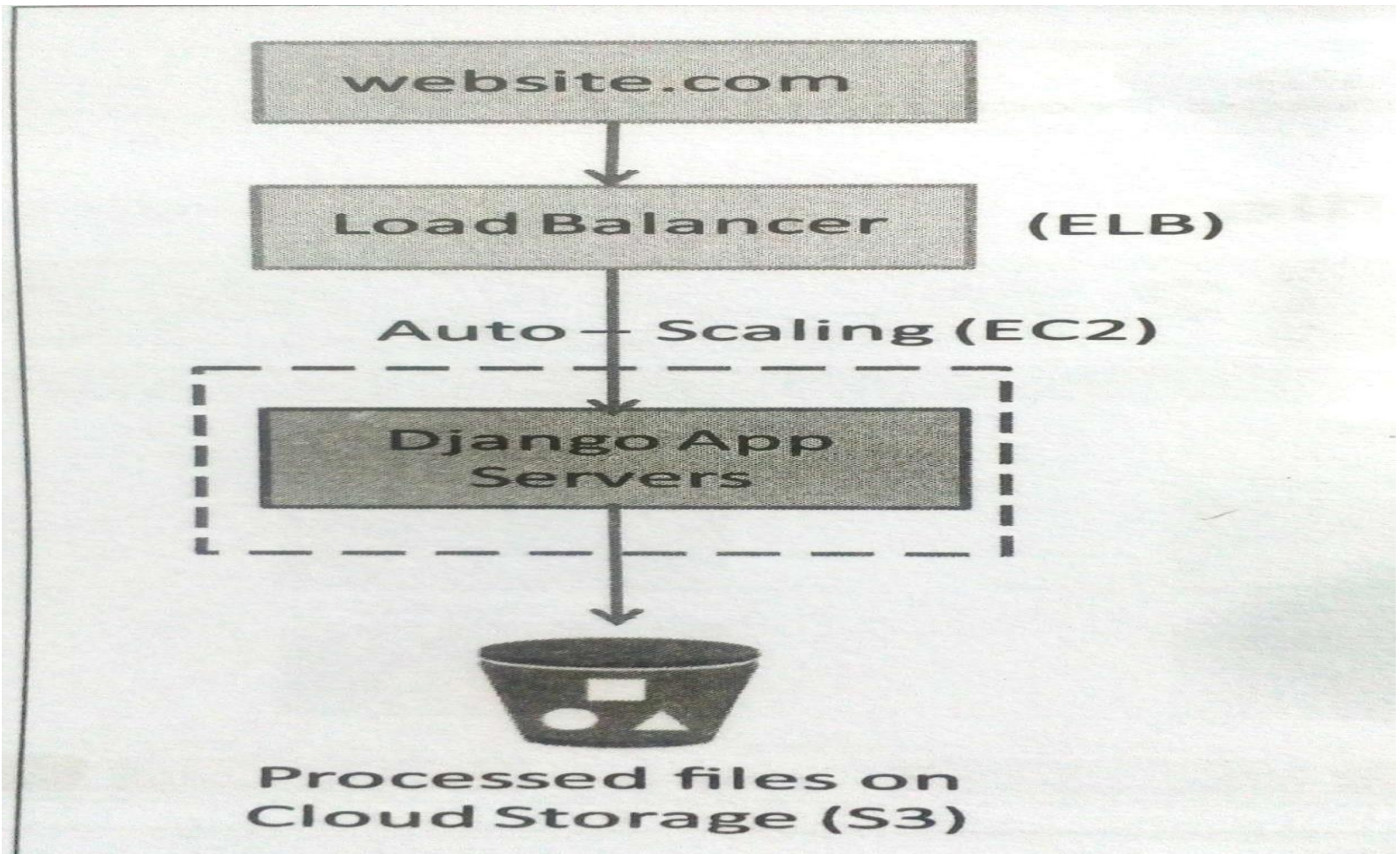


Fig : Deployment Design for IPA

Document Storage Application

- This application allows the user to **store documents** on a cloud-based storage.
- In the Component design step we identify the application components and group them based on the type of functions performed and type of resources required.
- The web tier for the cloud drive app has front **ends for uploading files**, viewing/deleting files and user profile.
- The application tier has components for processing requests for **uploading files, processing requests for viewing/deleting files and the** component that handles the registration, profile and login functions.
- The database tier comprises of a user credentials database.
- The storage tier comprises of the storage for files.
- In the architectural design step which defines the interactions between the application components.
- MySql is used for database tier and Cloud storage is used for the storage tier.

Document Storage Application

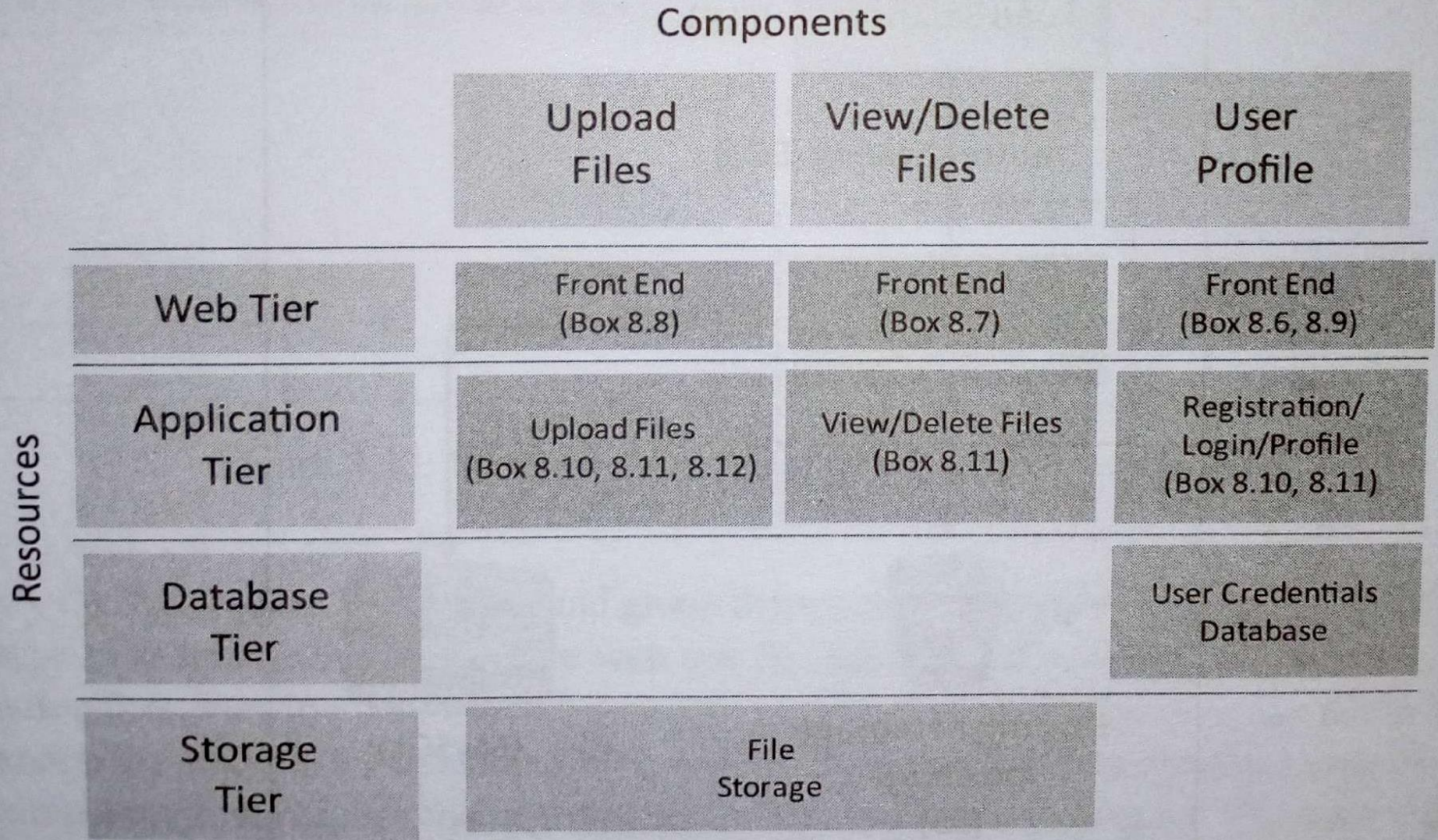


Fig : Component design for CDA
Clinical Document Architecture (CDA)

Document Storage Application

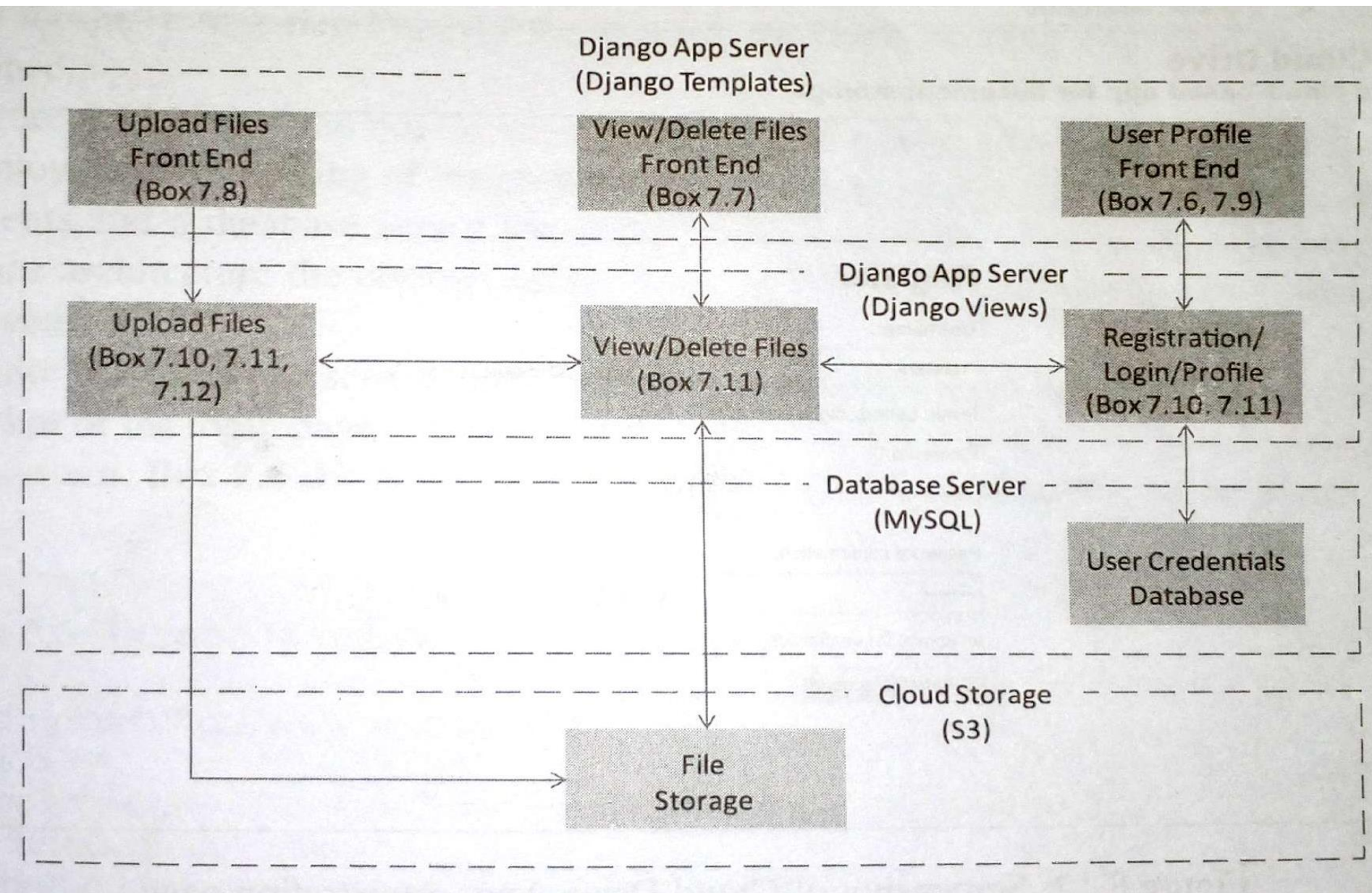


Fig : Architectural Design for CDA

Document Storage Application

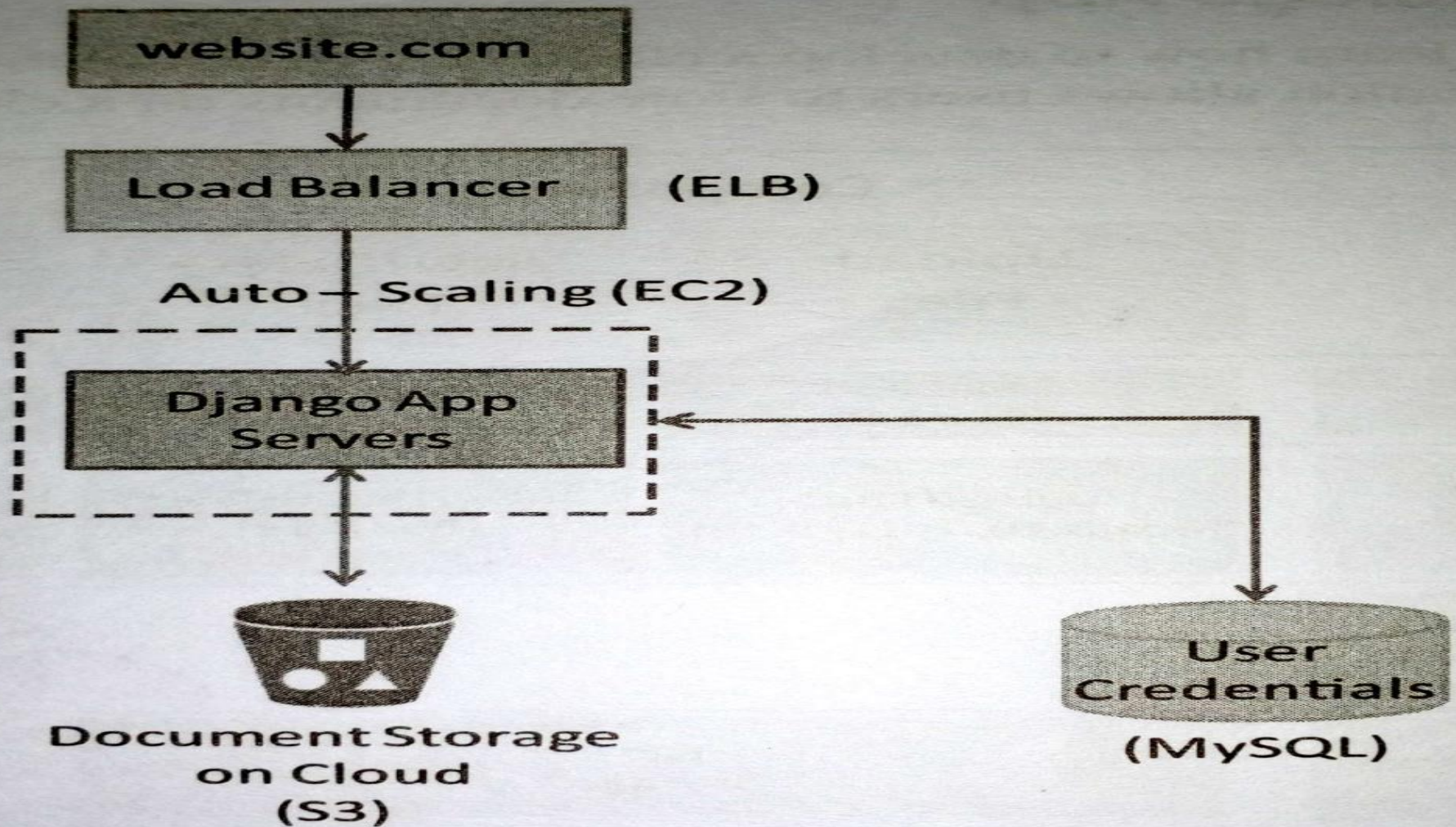


Fig :Deployment Design for CDA

Document Storage Application

- This is a multitier deployment comprising of load balancer, application servers, cloud storage for
- Storing documents and a database server for storing user credentials.

Map Reduce Application

- This application allows users to **submit MapReduce** jobs for data analysis.
- It is based on the **amazon elastic map reduce (EMR)** service.
- Users can upload data files to analyze and choose/upload the Map and reduce programs.
- The selected Map and Reduce programs along with the data are submitted to a **queue for processing**.
- In the component design step we identify the application components and group them based on the type of functions performed and type of resources required.
- The web **tier for the map reduce app has a front end** for MapReduce job submission.
- The application tier has components for processing requests for uploading files, **creating MapReduce jobs and enqueueing jobs**, MapReduce consumer and the component for email notifications.

Map Reduce Application

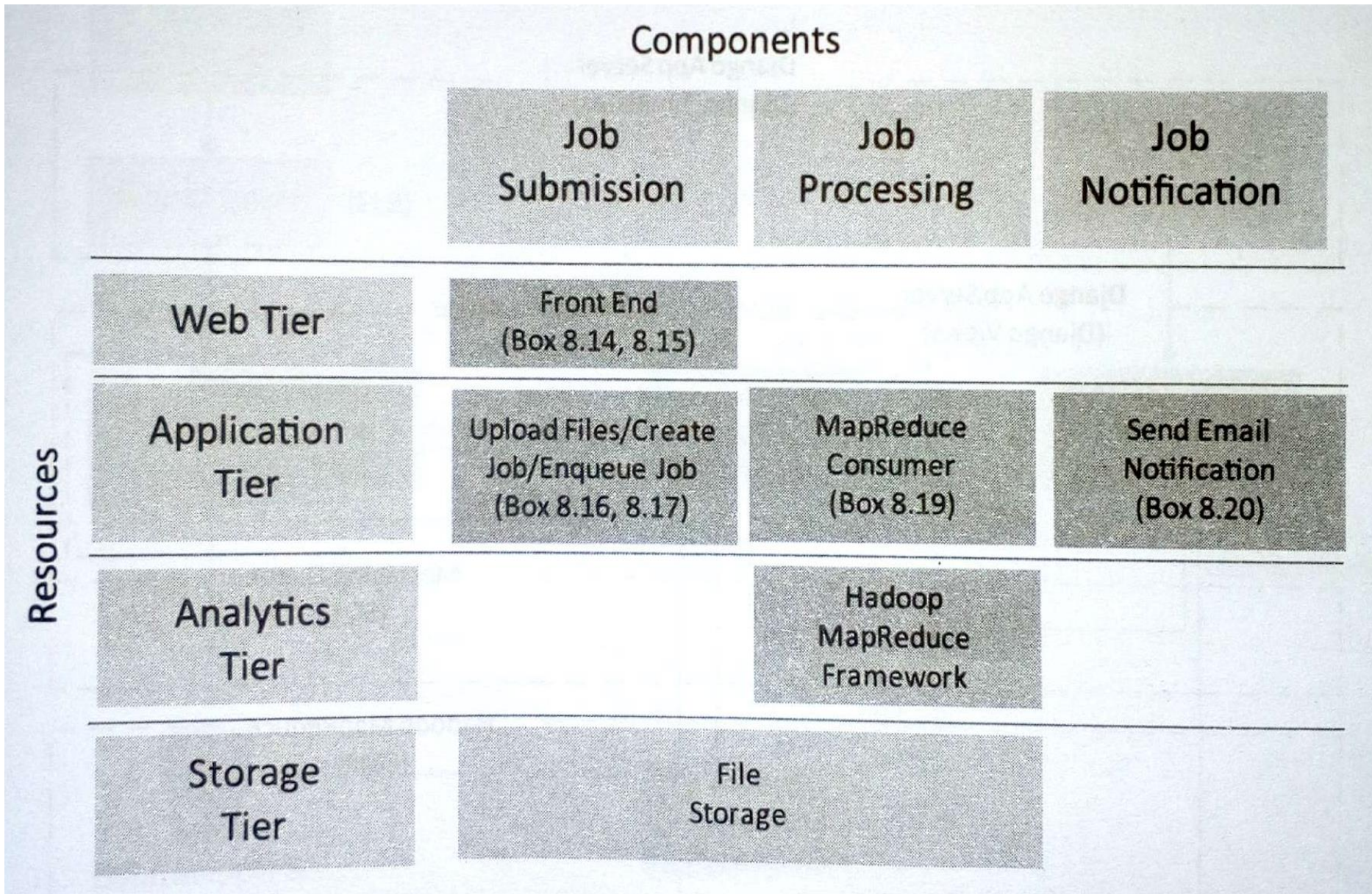


Fig Component Design for MapReduce

Map Reduce Application

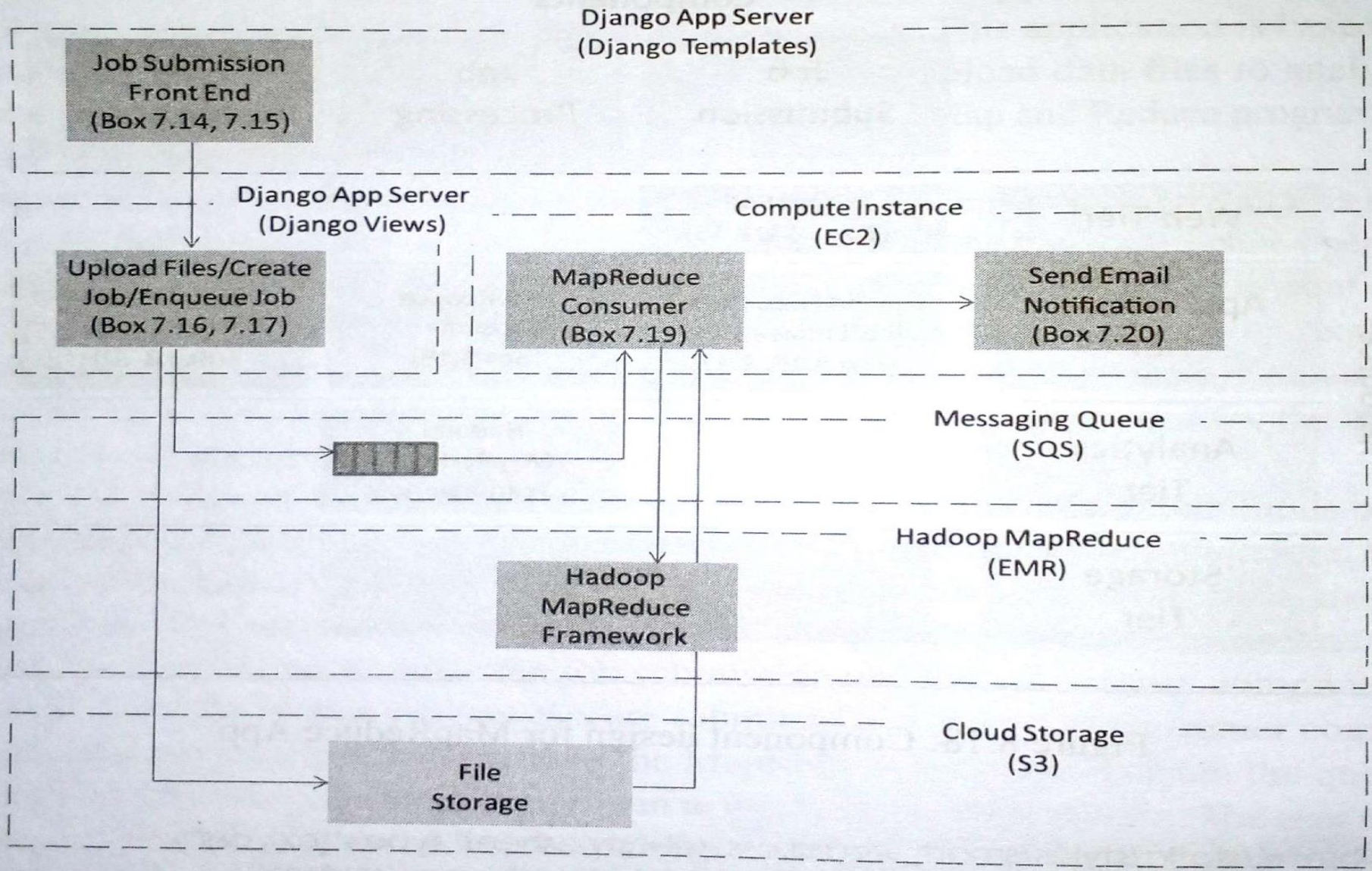


Fig: Architecture Design for Map Reduce App

Map Reduce Application

- The **Hadoop frame work** is used for the analytics tier and a cloud storage is used for the storage tier.
- Architectural **design step** defines the interactions between the application components.
- It uses **Django framework** , so that the web tier components map to the django templates and the application tier components map to the django views.
- Deployment design has a multitier architecture **comprising of load balancer, application servers** and a cloud storage for storing MapReduce programs, input data and MapReduce output.

Map Reduce Application

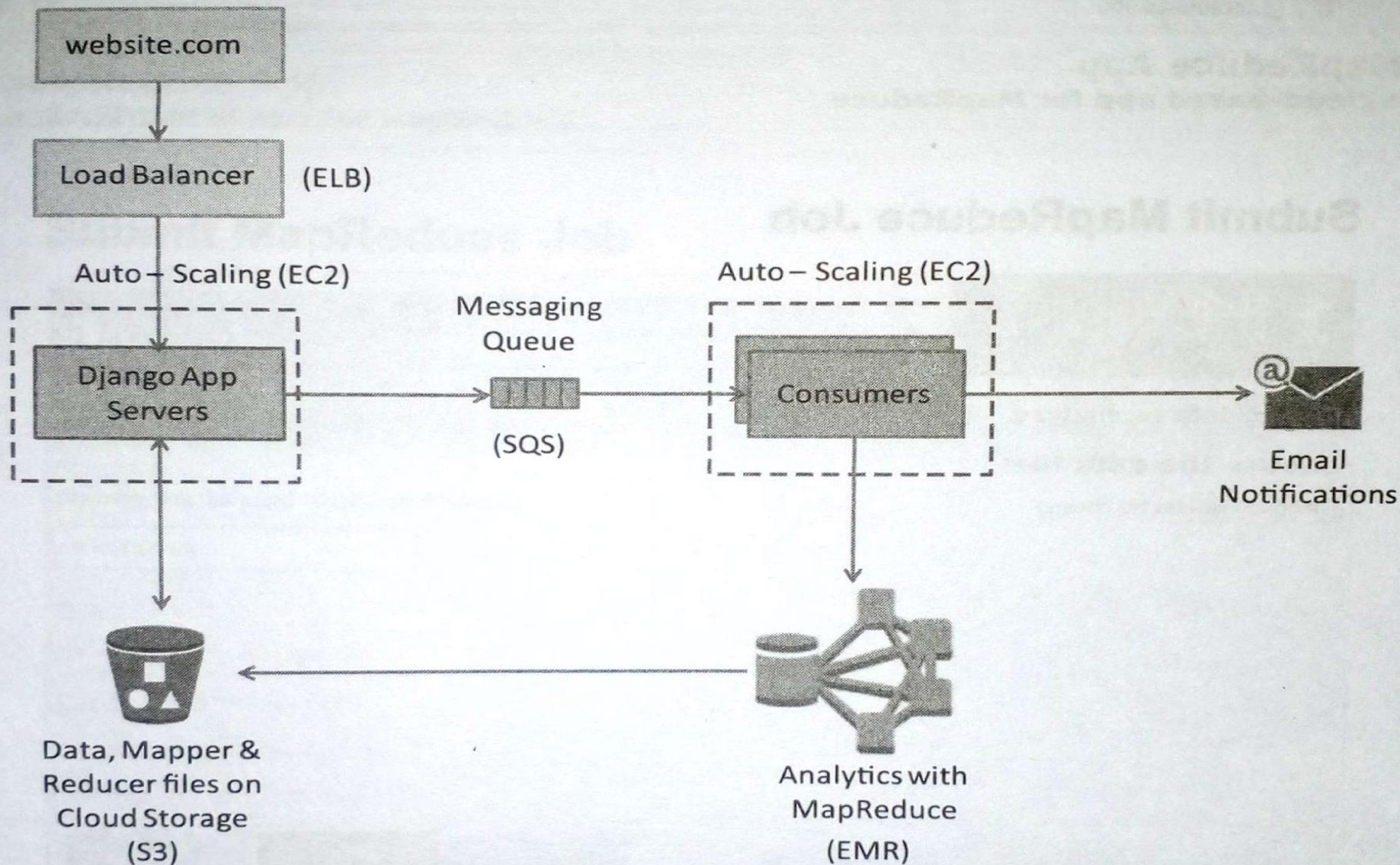


Fig: Deployment Design for Map Reduce App

Social Media Application

- This application collects the **social media feeds(tweets)** on a specified keyword in real time and analyzes the sentiments of tweets and provides aggregate results.
- In the component design phase we identify the application **components and group them based** on the type of functions performed and type of resources required.
- The web tier has a front end for displaying results.
- **The application tier has a listener component** that collects social media feeds, a consumer component that analyzes **tweets and a component for rendering the results in the dashboard.**
- We use a database for database tier and a cloud storage is used for the storage tier.
- In the architectural design we define the interactions between the application components.

Social Media Application

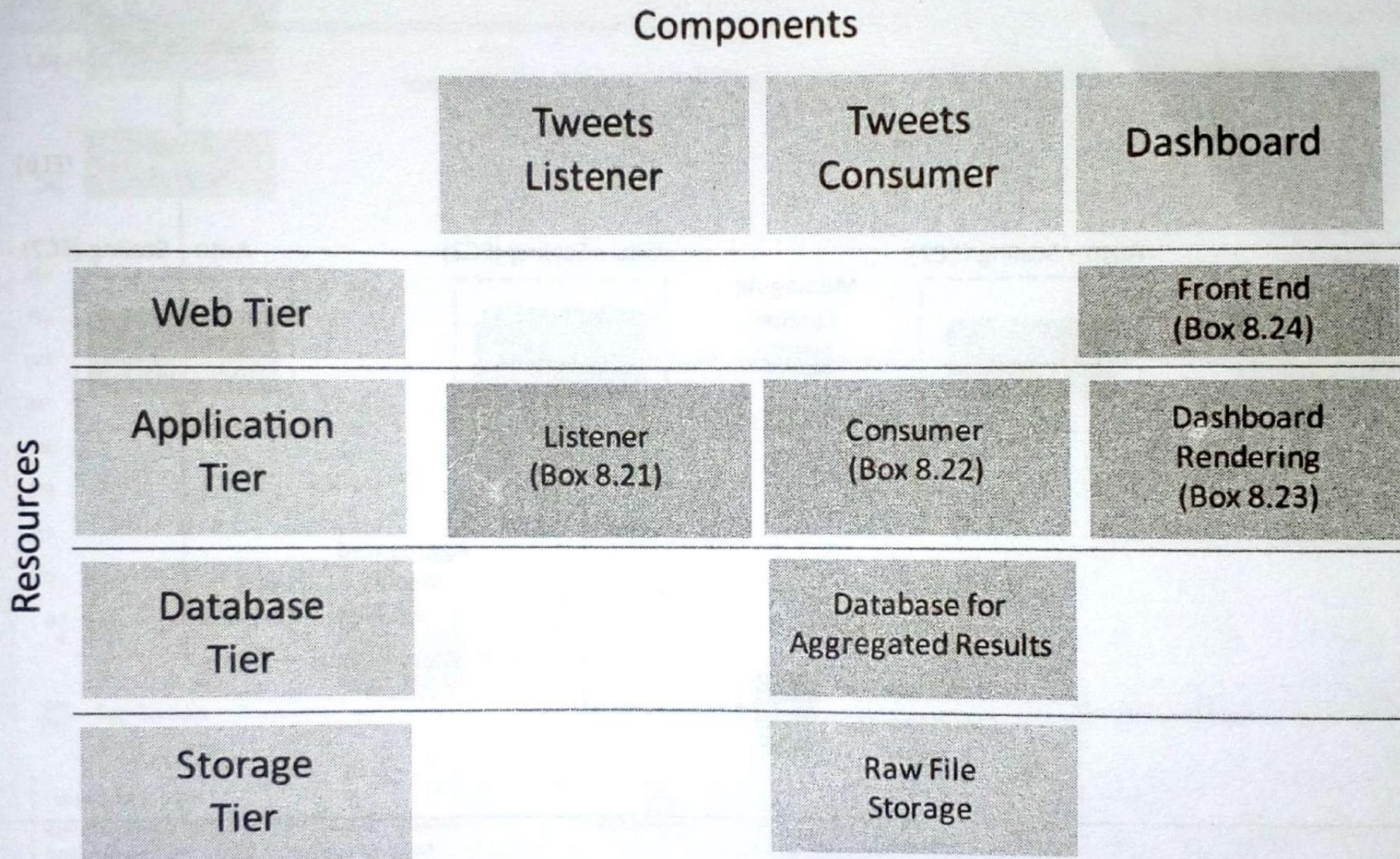


Fig : Component design for Social Media Analytics App

Social Media Application

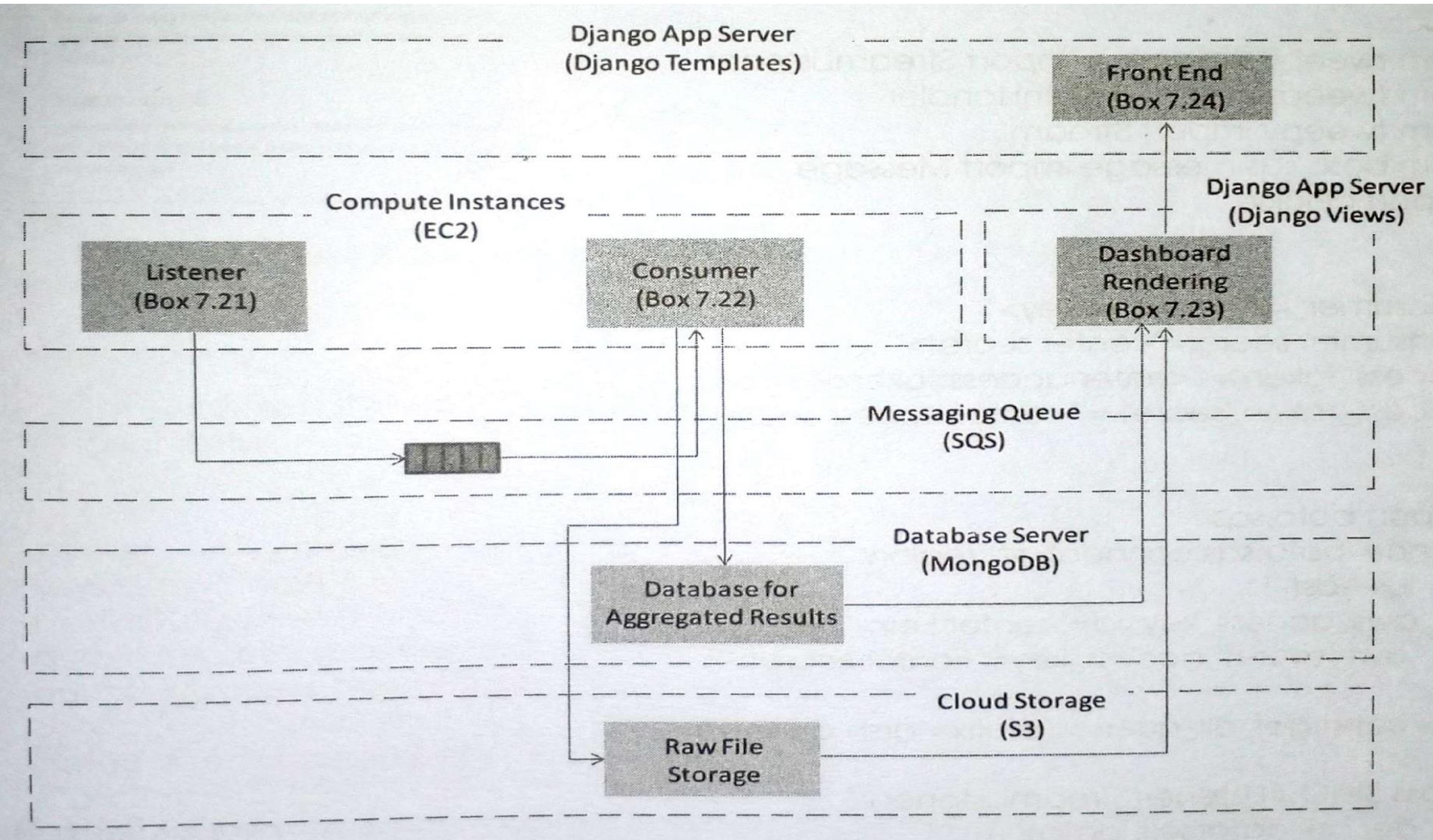


Fig : Architecture design for Social Media Analytics App

Social Media Application

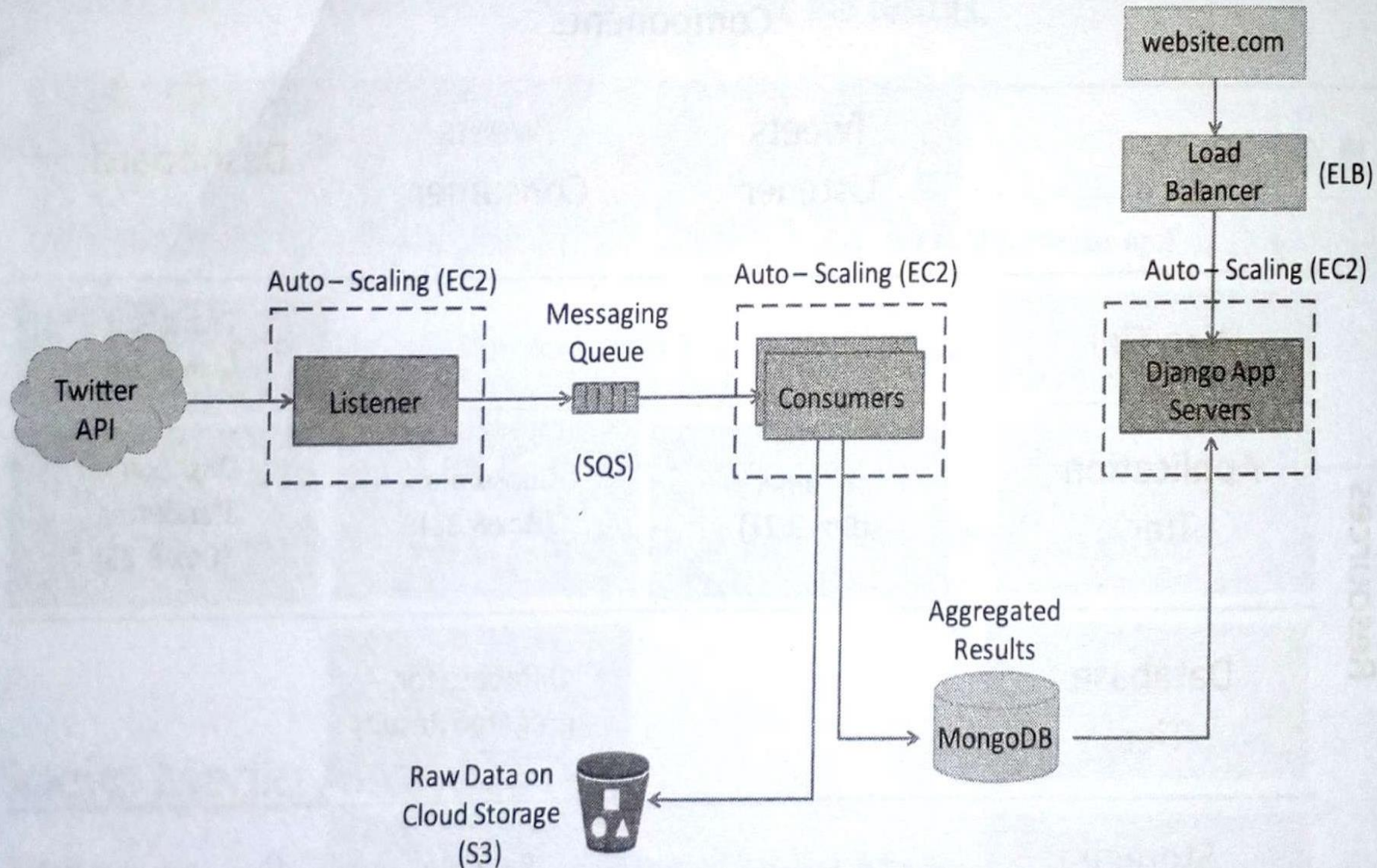


Fig : Deployment design for Social Media Analytics App

Social Media Application

- Deployment design is a multi-tier architecture comprising of load balancer, application servers, listener and consumer instances, cloud storage for storing raw data and a data base server for storing aggregated results.

Thank You